

FROM ABSTRACTIONS TO FIRST PRINCIPLES

The Topic Graph Curriculum

*Designed on Math Academy principles. No weeks. No schedules.
Master each topic. Prove it. Then unlock the next.*

61 topics | 2 automaticity gates | 6 retrieval exercises | 69 sessions total
Language: Python | Pace: mastery, not calendar

How This Curriculum Works

This curriculum is structured on the principles of Math Academy: atomic topics, prerequisite graphs, mastery gates, spaced retrieval, and interleaving. There are no weeks. There is no schedule. You move through topics at the pace of mastery.

The Six Principles

1. Atomic Topics. The curriculum is decomposed into 61 small topics, plus 6 retrieval exercises and 2 automaticity gates (69 sessions total). Each topic teaches one concept and requires one hands-on build task. Topics take 30–90 minutes. Some you'll clear fast. Others will take multiple sessions.

2. Prerequisite Graph. Every topic lists its prerequisites. You cannot start a topic until you've demonstrated mastery of its prerequisites. This is not a suggestion — it's a hard gate. Your partner verifies.

3. Mastery Checks. Every topic ends with a mastery check. This is not self-assessed. Your partner watches you explain the concept and demonstrate the skill. If you can't do it without looking at notes or code, you haven't mastered it.

4. Automaticity Gates. At two key points, you must rebuild from scratch without reference material. This proves that foundational skills are automatic — that they no longer consume working memory.

5. Spaced Retrieval. Six retrieval exercises are inserted throughout the curriculum. These are timed, no-notes challenges that force you to recall and apply earlier concepts. They fight the forgetting curve.

6. Interleaving. Topics from different domains are deliberately interleaved to avoid encoding interference. You don't do six replication topics in a row. You do replication, then observability, then back to replication. This feels harder but produces deeper learning.

The Topic Format

Every topic follows the same structure:

- **CONCEPT:** A brief explanation of the idea, the mental model, and why it matters. This is your “worked example.” Read it, understand it, but don't linger. 5–15 minutes.
- **BUILD:** A specific hands-on task. You write code. You run it. You observe behavior. This is where learning happens. 20–60 minutes.
- **MASTERY CHECK:** Your partner asks you to explain or demonstrate. No notes. No code reference. If you pass, the topic is complete and you unlock dependents. If not, revisit and try again.
- **REAL-SYSTEM CONNECTION (some topics):** A pointer to how a production system solves this same problem. Read after mastery, not before.

How to Track Progress

Create a shared spreadsheet or Notion board with all 69 sessions (61 topics + 6 retrievals + 2 gates). For each, track: Not Started → In Progress → Built → Mastery Verified (by partner). A

topic is only “done” when your partner has verified mastery. You each maintain your own board since you’re building separately.

Interleaving Order

The topics are listed in this document by domain for reference. But you should NOT work through them domain by domain. Follow the Recommended Path at the end of this document, which interleaves topics across domains while respecting prerequisites. The recommended path is your primary guide — the domain-organized listing is your reference.

Reading Material

Each topic references specific pages or chapters. The primary texts are:

- **DDIA** — Designing Data-Intensive Applications (Kleppmann)
- **Beej** — Beej’s Guide to Network Programming (free online)
- **CPython** — CPython Internals (Shaw) or Dave Beazley’s GIL talk
- **DSO** — Distributed Systems Observability (Sridharan)

Domain A: TCP & Sockets

These topics build your understanding of what happens at the OS level when two machines communicate. Everything else in the curriculum sits on top of this.

A1 Socket Creation and Binding

Prerequisites: None

CONCEPT

A socket is a file descriptor the OS gives you for network communication. `socket()` creates it, `bind()` attaches it to an address and port. The OS manages the underlying kernel buffer.

Read: Beej's Guide, sections 1–3.

BUILD

Create a Python script that: (1) creates a TCP socket using `socket.socket(AF_INET, SOCK_STREAM)`, (2) binds it to localhost on a port, (3) prints the file descriptor number. Then try binding two sockets to the same port. Observe the error. Try binding to port 0 and see what the OS assigns.

MASTERY CHECK — Your partner verifies

Explain to your partner: what is the difference between a socket and a connection? What does the OS do when you call `bind()`? What happens if the port is already in use, and why?

A2 Listening and Accepting Connections

Prerequisites: A1

CONCEPT

`listen()` tells the OS to queue incoming connections. `accept()` dequeues one and returns a new socket for that specific connection. The backlog parameter controls the queue size.

Read: Beej's Guide, sections 4–5.

BUILD

Extend A1: add `listen(backlog=5)` and `accept()`. Print the client address when a connection arrives. Test by connecting with: `python -c "import socket; s=socket.socket(); s.connect(('localhost', PORT)); print('connected')"`

MASTERY CHECK — Your partner verifies

Your partner connects 3 clients rapidly. Explain: what is the backlog queue? What happens to connection #6 if `backlog=5` and you haven't called `accept()` yet? Trace the kernel-level flow.

A3 Send and Receive Byte Streams

Prerequisites: A2

CONCEPT

TCP is a byte stream, not a message stream. `send()` pushes bytes into the kernel buffer. `recv(N)` reads UP TO N bytes — it may return fewer. There are no message boundaries in TCP.
Read: Beej's Guide, section 6.

BUILD

Build a server that accepts a connection and calls `recv(1024)`. Build a client that sends a 4096-byte message. Print how many bytes the server actually received on the first `recv()`. Run it 10 times. Observe that the number varies.

MASTERY CHECK — Your partner verifies

Explain: why does `recv(1024)` sometimes return fewer than 1024 bytes even when the client sent more? Where are the remaining bytes? What guarantees does TCP actually provide?

A4 Line-Based Message Framing

Prerequisites: A3

CONCEPT

Since TCP has no message boundaries, applications must define their own framing protocol. Line-based framing uses `\n` as a delimiter. You buffer incoming bytes and split on `\n`. This forces you to handle partial reads and multiple messages in one `recv()`.
This is your first protocol design decision.

BUILD

Build a `recv_line()` function that: reads bytes into a buffer, scans for `\n`, returns the complete line (without `\n`), and keeps remaining bytes in the buffer for next call. Build a server that echoes each line back. Test with a client that sends multiple lines in one `send()` call and also sends a line split across two `send()` calls.

MASTERY CHECK — Your partner verifies

Show your partner the case where one `send()` contains two lines. Show the case where one line arrives across two `recv()` calls. Explain why both cases require buffering. Write `recv_line()` from memory on a whiteboard or blank file.

A5 Connection Lifecycle and Cleanup

Prerequisites: A4

CONCEPT

Connections must be properly closed. `recv()` returning 0 bytes means the peer closed their end. Failing to close sockets leaks file descriptors. The OS has a limit (`ulimit -n`). `TIME_WAIT` state means a closed socket's port isn't immediately reusable. `SO_REUSEADDR` solves the `TIME_WAIT` problem during development.

BUILD

Add proper connection cleanup to your server: detect `recv()=0`, close the socket, print that the client disconnected. Add `SO_REUSEADDR`. Test by rapidly stopping and restarting your server. Without `SO_REUSEADDR`, observe the "Address already in use" error.

MASTERY CHECK — Your partner verifies

Explain: what does `recv()` returning 0 bytes mean versus `recv()` raising an exception? What is `TIME_WAIT` and why does the OS keep it? What is a file descriptor leak and when does it become a problem?

A6 Socket Timeouts and Error Handling

Prerequisites: A5

CONCEPT

Network operations can hang indefinitely. `settimeout()` prevents this. Different errors mean different things: `ConnectionRefusedError` (nothing listening), `TimeoutError` (no response), `BrokenPipeError` (peer crashed). Your code must handle each case. Robust networking code handles all three.

BUILD

Add `socket.settimeout(5.0)` to your client. Test against a server that accepts but never responds. Test against a non-existent server. Test against a server that crashes mid-response. Handle each error type with a specific message. Log each error.

MASTERY CHECK — Your partner verifies

Your partner gives you three failure scenarios. You predict which exception will be raised and why, before running the code. Explain the difference between a timeout and a connection refusal at the OS level.

Domain B: HTTP

HTTP/1.1 is text over TCP. These topics make you build an HTTP/1.1 server by parsing raw bytes, which is what the protocol layer beneath web frameworks does.

B1 HTTP Request Line Parsing

Prerequisites: A4

CONCEPT

An HTTP request starts with a request line: `METHOD PATH VERSION\r\n`. Example: `GET /store/mykey HTTP/1.1\r\n`. You parse this by splitting on spaces. The method tells you what operation, the path tells you what resource.

Read: RFC 9112, section 3 (request line format).

BUILD

Using your line-based server from A4, modify `recv_line()` to handle `\r\n` instead of `\n`. Read the first line of an incoming HTTP request. Parse it into method, path, and version. Print each. Test with: `curl -v http://localhost:PORT/hello`

MASTERY CHECK — Your partner verifies

Parse a request line from memory. Explain: why does HTTP use `\r\n` and not just `\n`? What are three valid HTTP methods and their semantics?

B2 HTTP Header Parsing

Prerequisites: B1

CONCEPT

After the request line, headers follow as `Key: Value\r\n` pairs, terminated by an empty line (`\r\n\r\n`). Headers are case-insensitive. Important headers: `Host` (required in HTTP/1.1), `Content-Length` (tells you body size), `Content-Type`.

Read: RFC 9110, section 6.3 (header fields).

BUILD

After parsing the request line, read lines until you hit an empty line. Parse each into a key-value pair. Store in a dictionary (lowercase keys for case-insensitivity). Print all parsed headers. Test with `curl -v` and verify your parser matches what curl sends.

MASTERY CHECK — Your partner verifies

Your partner sends a request with unusual headers (mixed case, extra whitespace). Your parser handles it correctly. Explain: why is the Host header required? What breaks without it?

B3 HTTP Body Handling

Prerequisites: B2

CONCEPT

The body follows the header section. Content-Length tells you how many bytes to read. Without it, you don't know when the body ends. Read exactly Content-Length bytes after the headers. Not more, not less.

Read: RFC 9112, section 6 (message body).

BUILD

If Content-Length header is present, read that many bytes from the socket after the empty line. Store as the request body. Test with: `curl -X PUT -d 'hello world' http://localhost:PORT/test` and verify your server received exactly "hello world".

MASTERY CHECK — Your partner verifies

Explain: what happens if you read fewer bytes than Content-Length? What if you read more? What if the client doesn't send Content-Length with a PUT?

B4 HTTP Response Construction

Prerequisites: B3

CONCEPT

An HTTP response is: STATUS_LINE\r\n HEADERS\r\n\r\n BODY. Status line: HTTP/1.1 STATUS_CODE REASON\r\n. You must include Content-Length and Content-Type headers. Use `conn.sendall()` (not `send()`) to transmit the response — `send()` may write only part of the buffer, while `sendall()` guarantees the full payload is sent.

Read: RFC 9110, section 15 (status codes).

BUILD

Build a `send_response(conn, status_code, body, content_type)` function that constructs and sends a well-formed HTTP response. Return 200 for success, 404 for not found, 400 for bad request. Test with `curl -v` and verify response headers.

MASTERY CHECK — Your partner verifies

Write `send_response()` from memory. Explain: what is the difference between 200, 201, 400, 404, and 500? When would you use each?

B5 Routing — Path Matching and Method Dispatch

Prerequisites: B4

CONCEPT

Routing maps (METHOD, PATH) pairs to handler functions. You match the path using string parsing. Dynamic segments like `/store/<key>` require extracting the key from the path. A router is just a dictionary of patterns to functions.

This is the core of what Flask/FastAPI does.

BUILD

Build a simple router: a dictionary mapping (method, path_pattern) to handler functions. Handle GET `/health`, PUT `/store/<key>`, GET `/store/<key>`. Extract `<key>` from the path. Return 404 for unrecognized routes. Return 405 for wrong method on a known path.

MASTERY CHECK — Your partner verifies

Add a new route GET `/stats` without looking at your existing routing code for reference. Explain: how does FastAPI's path parameter syntax (`/items/{item_id}`) map to what you just built?

B6 The KV Store HTTP Interface

Prerequisites: B5

CONCEPT

Combine everything into a working HTTP key-value store. PUT `/store/<key>` with a body stores the value. GET `/store/<key>` retrieves it. GET `/health` returns 200. Store data in a Python dictionary for now.

This is your first complete working system.

BUILD

Wire up the router to actual handlers that read/write a Python dictionary. Test the full flow: PUT a value, GET it back, GET a non-existent key (404). Test with curl and verify every response is well-formed HTTP.

MASTERY CHECK — Your partner verifies

Your partner runs a sequence of 5 curl commands against your server. Every response is correct with proper status codes, headers, and bodies. Explain: is this server stateless or stateful? Why?

REAL-SYSTEM CONNECTION

Read: uvicorn's HTTP parsing layer. Uvicorn uses h11 (or httptools) to handle the HTTP protocol, not Starlette. Starlette is the ASGI framework on top. Look at h11's source (github.com/python-hyper/h11) and compare its request parsing to yours.

Domain C: Storage

These topics replace the in-memory dictionary with durable storage that survives crashes. You're building the simplest possible version of what databases do.

C1 File I/O Fundamentals

Prerequisites: B6

CONCEPT

Python's `open()`, `write()`, `read()`, and `seek()` map to OS-level system calls. The OS maintains a file position pointer. Writing appends at the current position. Seeking moves the pointer. Modes: 'a' for append, 'r' for read, 'rb'/'wb' for binary.

Read: DDIA Chapter 3, pages 69–72 (hash indexes intro).

BUILD

Write a script that: (1) opens a file in append mode, (2) writes 5 key=value lines, (3) closes it, (4) reopens in read mode, (5) reads all lines and prints them. Then use `seek()` to jump to a byte offset and read from there. Print the byte offset of each line.

MASTERY CHECK — Your partner verifies

Explain: what is the difference between the OS's file buffer and the Python file object's buffer? When does data actually reach the disk?

C2 Append-Only Log

Prerequisites: C1

CONCEPT

An append-only log is the simplest durable storage: every write appends a new entry to a file. To find a key's latest value, scan from the end. Deletes are recorded as tombstone entries. This is how real databases begin.

Read: DDIA Chapter 3, pages 72–75 (log-structured storage).

BUILD

Replace your in-memory dictionary with an append-only log file. Format: each entry is `key\tvalue\n` (tab-separated). PUT appends a new entry. GET scans the file from the end to find the most recent entry for a key. DELETE appends `key\t__TOMBSTONE__\n`.

MASTERY CHECK — Your partner verifies

PUT 3 different keys, then PUT an updated value for the first key. Show your partner the log file. Explain: why does the file have 4 entries, not 3? How does GET find the right value?

C3 In-Memory Index Over Log

Prerequisites: C2

CONCEPT

Scanning the log for every GET is $O(n)$. An in-memory hash index maps each key to its byte offset in the log file. GET becomes: look up offset in the index, seek to that offset, read the entry. $O(1)$.

Read: DDIA Chapter 3, pages 72–74 (hash table index for log-structured storage).

BUILD

Maintain a Python dictionary mapping keys to byte offsets. When you write an entry, record the byte offset before writing. When you GET, look up the offset, seek to it, read the entry. Measure: time 10,000 GETs with and without the index.

MASTERY CHECK — Your partner verifies

Explain: the index is in memory. What happens if the server crashes? How do you rebuild the index? What is the time complexity of a rebuild?

C4 Crash Recovery

Prerequisites: C3

CONCEPT

When the server starts, it must rebuild the index by scanning the entire log. But what if the server crashed mid-write? The last entry might be corrupted (partial write). You need to detect and handle this.

A corrupted last entry can be detected by checking for a trailing newline.

BUILD

Implement a `rebuild_index()` function that runs on startup: scan the log from the beginning, build the index entry by entry. If the last entry is incomplete (no trailing `\n`), discard it. Add a startup log message showing how many entries were recovered.

MASTERY CHECK — Your partner verifies

Kill your server (Ctrl+C or kill -9) while it's processing a write. Restart it. Does it recover correctly? Explain: what data can you lose in this scenario and why? What would you need to guarantee zero data loss?

C5 fsync and Durability

Prerequisites: C4

CONCEPT

`file.write()` puts data in the OS buffer cache. It may not be on disk yet. `os.fsync(file.fileno())` forces the OS to flush to disk. Without `fsync`, a power failure can lose recent writes even if your program didn't crash.

Read: DDIA Chapter 3, pages 82–83 (making B-trees reliable — the WAL discussion applies).

BUILD

Add `os.fsync()` after every write. Measure the performance: time 1000 PUTs with `fsync` vs without. The difference should be dramatic (10–100x). Then try `fsync` every 100 writes as a compromise. Measure again.

MASTERY CHECK — Your partner verifies

Explain to your partner: where is your data after `write()` but before `fsync()`? Draw the path: Python buffer → OS page cache → disk. What failure scenarios does `fsync` protect against that `write()` alone doesn't?

REAL-SYSTEM CONNECTION

Read: “The Fundamentals of RocksDB” (getstream.io/blog/rocksdb-fundamentals). Compare RocksDB's WAL (Write-Ahead Log) to your append-only log. They solve the same durability problem.

C6 Log Compaction

Prerequisites: C5

CONCEPT

The log grows forever because updates append new entries without removing old ones. Compaction creates a new log containing only the latest entry for each key, then replaces the old log. This reclaims space.

Read: DDIA Chapter 3, pages 73–74 (compaction and merging).

BUILD

Implement `compact()`: scan the current log, build a dictionary of latest values, write them to a new file, atomically replace the old file (`os.replace()` — not `os.rename()`, which fails on Windows if the destination exists), rebuild the index. Run compaction when the log exceeds a size threshold (e.g., 1MB).

MASTERY CHECK — Your partner verifies

Your log has 10,000 entries but only 500 unique keys. After compaction, how many entries remain? Explain: why use `os.replace()` for atomic file replacement? What happens if the server crashes during compaction?

Domain D: Concurrency

Your server handles one client at a time. These topics fix that and force you to understand Python's threading model.

D1 The GIL — What It Is and Why

Prerequisites: C5

CONCEPT

The Global Interpreter Lock (GIL) means only one thread executes Python bytecode at a time. CPython uses it because Python's memory management (reference counting) isn't thread-safe. The GIL is released during I/O operations (socket, file), which is why threaded servers work for I/O-bound workloads.

Read: CPython Internals GIL chapter or Dave Beazley's GIL talk (~45 min).

BUILD

Write a script that spawns 4 threads, each computing `sum(range(10_000_000))`. Time it. Then time the single-threaded version. The threaded version should be SLOWER (GIL contention). Now do the same with 4 threads each doing socket I/O (connect to a slow server). The threaded version is faster. Explain why.

MASTERY CHECK — Your partner verifies

Explain: why does CPython have the GIL? What would break without it? When is the GIL released? Why do threaded web servers work despite the GIL?

D2 Thread-Per-Connection Server

Prerequisites: D1, A6

CONCEPT

Spawn a new thread for each accepted connection. Each thread handles one client independently. This is the simplest concurrent server model. The main thread just loops on `accept()`. This is how many production servers worked before async I/O.

BUILD

Modify your server: the main loop calls `accept()`, then spawns a `threading.Thread` to handle the connection. The main loop immediately goes back to `accept()`. Test with 10 simultaneous curl requests. Verify they all succeed.

MASTERY CHECK — Your partner verifies

Your partner opens 50 simultaneous connections. Your server handles all of them. Explain: what is the maximum number of simultaneous connections your server can handle? What limits it?

D3 Thread Pool with Bounded Workers

Prerequisites: D2

CONCEPT

Unlimited threads are dangerous: each thread consumes stack memory (the default varies by platform — often several MB). A thread pool limits concurrency. Use `concurrent.futures.ThreadPoolExecutor` or build your own with a queue. When all workers are busy, new connections wait.

This is what `uvicorn` and `gunicorn` do.

BUILD

Replace thread-per-connection with a `ThreadPoolExecutor(max_workers=10)`. Submit each connection to the pool. Test: open 15 connections when pool size is 10. Verify that 10 are served immediately and 5 wait. Measure queue wait time.

MASTERY CHECK — Your partner verifies

Explain: what happens to connection #11 when the pool has 10 workers? How long does it wait? What would happen in production if your pool is too small? Too large?

REAL-SYSTEM CONNECTION

Read: `uvicorn`'s source code on GitHub — the main server loop. `Uvicorn` uses an event loop (`asyncio`), not a thread pool. `Gunicorn` is a pre-fork process manager with multiple worker types. Compare both models to your thread pool approach. Understand why `FastAPI` chose `async` via `uvicorn`.

D4 Shared State and Race Conditions

Prerequisites: D3, C3

CONCEPT

Your in-memory index and log file are shared across threads. While single dict operations in CPython are effectively atomic under the GIL, the real danger is compound operations: a thread reads the index, another thread writes to the log and updates the index between the read and a subsequent write, causing lost updates or mismatched file/index state. The race condition is in multi-step operations, not individual dict calls.

This is one of the hardest bugs to find because it's non-deterministic.

BUILD

Write a script that spawns 100 threads, each doing a read-modify-write: GET a counter key, increment it, PUT it back. Without locking, the final counter value will be less than 100 (lost updates). With `threading.Lock()` around the full read-modify-write sequence, it's exactly 100. This demonstrates the compound operation problem, not single-dict-operation corruption.

MASTERY CHECK — Your partner verifies

Explain: what is a race condition? Draw a timeline showing two threads interleaving a read-modify-write that causes a lost update. Why is this bug hard to reproduce? Why doesn't the GIL protect you here?

D5 Read-Write Locks

Prerequisites: D4

CONCEPT

A mutex (Lock) blocks all access — even multiple simultaneous readers. A read-write lock (RWLock) allows multiple readers OR one writer. For a KV store with many reads and few writes, this dramatically improves throughput.

Python doesn't have a built-in RWLock, so you'll build one from threading primitives.

BUILD

Implement a simple RWLock class using `threading.Condition`. Allow multiple readers to acquire simultaneously, but writers get exclusive access. Replace your Lock with the RWLock. Measure: time 1000 concurrent reads with Lock vs RWLock. The RWLock should be significantly faster.

MASTERY CHECK — Your partner verifies

Explain: why is a RWLock better than a mutex for a read-heavy workload? When would a simple mutex be better? What happens if a writer is waiting while readers keep arriving (writer starvation)?

D6 Measuring Concurrent Performance

Prerequisites: D5

CONCEPT

You can't optimize what you don't measure. `time.perf_counter()` gives high-resolution timing. Measure: requests per second, average latency, p99 latency. Run load from a simple Python script that opens N concurrent connections.

These numbers are your system's baseline.

BUILD

Write a benchmark script: spawn N threads, each sending 100 requests (mix of PUTs and GETs). Record latency of each request. Calculate: total throughput (req/sec), average latency, p50, p95, p99 latency. Run with N=1, 10, 50, 100. Plot how throughput and latency change.

MASTERY CHECK — Your partner verifies

State your system's numbers from memory. "My single-node KV store handles X req/sec at p99 of Y ms with Z concurrent clients, bottlenecked by [specific thing]." Your partner should believe these numbers because you measured them.

AUTOMATICITY GATE: Rebuild the Single-Node KV Store from Scratch

Before touching distributed systems, your foundational skills must be automatic. This gate proves it. You will rebuild the entire single-node KV store — TCP server, HTTP parser, append-only log with index, thread pool — from scratch, without looking at any of your previous code, notes, or reference material.

GATE REQUIREMENTS

Close all code editors and browser tabs. Open a blank Python file.

Rebuild: TCP socket server with accept loop → HTTP request parser (request line, headers, body) → HTTP response builder → router with GET/PUT/health routes → append-only log with in-memory index → crash recovery on startup → thread pool for concurrent connections.

Time limit: 3 hours. If you finish in under 2 hours, your foundations are truly automatic.

Test: your partner runs 10 curl commands against your rebuilt server. All responses must be correct.

If you cannot complete this gate, return to the topics where you got stuck. You are not ready for distributed systems.

Domain E: Cluster Fundamentals

You now have a single-node system that is automatic. These topics make it aware of other nodes.

E1 Multi-Node Configuration

Prerequisites: Gate 1

CONCEPT

A cluster needs to know its members. A configuration file (JSON) lists each node's ID, host, and port. Each node reads this at startup and knows the full cluster topology. Start simple: static membership. Read: DDIA Chapter 5, pages 151–155 (leaders and followers intro).

BUILD

Create a `config.json` listing 3 nodes on localhost with different ports. Modify your server to read this config at startup. Store the cluster topology in memory. Add a `GET /cluster` endpoint that returns the list of known nodes and their status.

MASTERY CHECK — Your partner verifies

Your partner changes the config to 5 nodes. Your server handles it without code changes. Explain: what is the difference between static membership and dynamic discovery? What are the trade-offs?

E2 Node-to-Node HTTP Client

Prerequisites: E1

CONCEPT

Nodes need to talk to each other. Build an internal HTTP client using raw sockets — not the requests library. This means: open a socket, send a well-formed HTTP request, parse the response. You already know how to do all of this from domains A and B. This is where your earlier automaticity pays off.

BUILD

Build a function `send_request(host, port, method, path, body=None)` that: creates a socket, sends an HTTP request, reads and parses the response, returns `(status_code, body)`. Test by having node-1 send a `GET /health` to node-2.

MASTERY CHECK — Your partner verifies

Your partner asks you to write `send_request()` from scratch. You can do it because you already built both sides (client and server) in domains A and B. This should feel easy. If it doesn't, your foundations need more practice.

REAL-SYSTEM CONNECTION

Read: The Dynamo Paper, Sections 1–2. Understand Amazon's design goals for distributed key-value stores.

E3 Health Checking and Failure Detection

Prerequisites: E2

CONCEPT

Each node periodically sends a health check (`GET /health`) to every other node. Track which nodes responded and which timed out. A node is "suspected down" after *N* consecutive timeouts. False positives happen when the network is slow but not dead.

Read: DDIA Chapter 8, pages 277–282 (unreliable networks, detecting faults).

BUILD

Add a background thread that, every 2 seconds, pings all other nodes. Track each node's status: UP (last ping succeeded), SUSPECTED (1–2 consecutive failures), DOWN (3+ consecutive failures). Log status changes. Test by starting 3 nodes, then killing one.

MASTERY CHECK — Your partner verifies

Your partner adds artificial latency (`time.sleep`) to one node's `/health` endpoint. Your health checker may incorrectly mark it as down. Explain: what is a false positive in failure detection? How would you reduce false positives without being too slow to detect real failures?

E4 Cluster State Tracking

Prerequisites: E3

CONCEPT

Cluster state is the current view of which nodes are alive, what role each plays, and what data each owns. This must be consistent enough to make routing decisions but doesn't need to be perfectly synchronized across nodes.

Each node maintains its own view of the cluster.

BUILD

Create a ClusterState class that tracks: node list, each node's status (UP/SUSPECTED/DOWN), and the current leader (None for now). Expose GET /cluster/state that returns this as JSON. Verify that all 3 nodes have consistent views after startup.

MASTERY CHECK — Your partner verifies

Start 3 nodes. Kill node-2. Wait for health checks to detect it. Query GET /cluster/state on nodes 1 and 3. Are their views consistent? Explain: can the views ever be inconsistent? When?

R1 TCP & HTTP Retrieval

TIMED RETRIEVAL — No notes, no code reference

10-minute time limit. No notes, no previous code.

Task: Write a Python function that opens a TCP socket, sends a properly formatted HTTP PUT request with a JSON body, reads the response, and returns the status code.

If you complete this easily, your Domains A and B knowledge is solid.

If you struggle, revisit topics A3, A4, B1–B4 before continuing.

Domain F: Replication

These topics make your data survive node failures by copying it across multiple nodes.

F1 Leader Designation and Write Forwarding

Prerequisites: E4

CONCEPT

In single-leader replication, one node is the leader. All writes go to the leader. If a client sends a write to a follower, the follower forwards it to the leader (or rejects it). Reads can go to any node.

Read: DDIA Chapter 5, pages 152–158 (leader-based replication).

BUILD

Hardcode node-1 as leader in config. When a follower receives a PUT, it forwards the request to the leader using your `send_request()` function. The leader processes it and responds. The follower returns the leader's response to the client.

MASTERY CHECK — Your partner verifies

Your partner sends a PUT to a follower. Trace the full request path: client → follower → leader → response back. Explain: why not let all nodes accept writes?

F2 Synchronous Replication

Prerequisites: F1

CONCEPT

After the leader writes locally, it sends the write to all followers and WAITS for all of them to acknowledge before responding to the client. This guarantees all replicas have the data before the client hears “success.” But it’s slow.

Read: DDIA Chapter 5, pages 153–155 (synchronous vs asynchronous).

BUILD

After the leader writes to its own log, it sends the write to each follower. Wait for all followers to respond 200. Only then respond to the client. Measure the latency of a PUT with 2 followers. Compare to single-node latency.

MASTERY CHECK — Your partner verifies

State the measured latency difference. Explain: if one follower is slow, what happens to every client’s write latency? Why is this a problem at scale?

F3 Asynchronous Replication

Prerequisites: F2

CONCEPT

The leader responds to the client immediately after writing locally, then replicates to followers in the background. This is faster but means followers may lag behind the leader.

Most production databases use asynchronous replication.

BUILD

Modify your replication: after writing locally, spawn a background thread to replicate to followers. Respond to the client immediately. Measure: latency should drop back to near single-node levels.

MASTERY CHECK — Your partner verifies

Explain: what data can you lose if the leader crashes after responding to the client but before replication completes? Is this acceptable? When?

F4 Measuring Replication Lag

Prerequisites: F3

CONCEPT

Replication lag is the time between a write hitting the leader and that write being available on a follower. With async replication, lag is nonzero. You can measure it by comparing timestamps or sequence numbers.

Read: DDIA Chapter 5, pages 161–167 (problems with replication lag).

BUILD

Add a write sequence number to each log entry. Each follower tracks its latest applied sequence number. Add GET /replication/status on each follower that returns its latest sequence number. The leader tracks its own. The difference is the lag. Add this to your /metrics endpoint.

MASTERY CHECK — Your partner verifies

Your partner sends 100 rapid writes. Immediately query each follower's replication status. What is the typical lag in sequence numbers? In milliseconds? Explain: what causes the lag?

REAL-SYSTEM CONNECTION

Read: "MySQL Replication Internals" by Arpit Bhayani (arpitbhayani.me). Compare MySQL's binlog position tracking to your sequence numbers. They solve the same problem.

F5 Follower Catches Up After Restart

Prerequisites: F4

CONCEPT

If a follower was down and comes back, it's behind. It needs to request all writes it missed from the leader. Using sequence numbers, the follower tells the leader "I'm at sequence N, send me everything after that."

This is called "catch-up recovery."

BUILD

Add a GET /replication/log?after=N endpoint to the leader that returns all log entries after sequence number N. When a follower starts up, it queries its own latest sequence number, then calls this endpoint to get missed entries. Apply them in order.

MASTERY CHECK — Your partner verifies

Kill a follower. Send 50 writes to the leader. Restart the follower. Verify it catches up automatically. Your partner queries the follower for a key that was written while it was down. It must return the correct value.

REAL-SYSTEM CONNECTION

Read: PlanetScale's blog on MySQL replication (planetscale.com). Note how GTIDs solve the same catch-up problem.

R2 Storage & Concurrency Retrieval

TIMED RETRIEVAL — No notes, no code reference

15-minute time limit. No notes, no previous code.

Task 1: Write an append-only log storage class with `put(key, value)`, `get(key)`, and `rebuild_index()` methods. Include `fsync`.

Task 2: Explain the GIL to your partner. Then explain why a read-write lock is better than a mutex for a read-heavy KV store.

If you struggle with Task 1, revisit C2–C5. If you struggle with Task 2, revisit D1, D5.

Domain G: Failure Handling

Things break. These topics teach you to handle it.

G1 Leader Failure Detection

Prerequisites: F4, E3

CONCEPT

Followers use health checks to detect when the leader is down. After N consecutive health check failures, a follower marks the leader as DOWN. This triggers the election process. The tricky part: distinguishing a dead leader from a slow leader.

Read: DDIA Chapter 8, pages 277–282 (timeouts and failure detection).

BUILD

When the health checker marks the leader as DOWN, log a critical message and trigger the election process (G2). Test by killing the leader process. Verify that followers detect it within 2–3 health check intervals.

MASTERY CHECK — Your partner verifies

Explain: what is the danger of setting the timeout too low? Too high? What is the optimal timeout and why is there no perfect answer?

G2 Basic Leader Election

Prerequisites: G1

CONCEPT

When the leader is detected as down, surviving nodes elect a new leader. Simple approach: the node with the lowest ID becomes leader. This is deterministic and requires no coordination protocol. But it has a flaw you'll discover.

Read: DDIA Chapter 5, pages 156–158 (handling node outages — failover).

BUILD

When a follower detects the leader is DOWN: if this node has the lowest ID among surviving nodes, promote self to leader and start accepting writes. Notify other nodes via a POST /cluster/new-leader message. Other nodes update their cluster state.

MASTERY CHECK — Your partner verifies

Your partner asks: what if the node with the lowest ID has stale data (it missed some writes before the old leader died)? Explain why this is a problem and propose a fix (hint: the node with the highest sequence number should become leader).

G3 Term Numbers and Fencing

Prerequisites: G2

CONCEPT

A term number is a monotonically increasing counter. Each leader election increments the term. Every write includes the term it was issued under. Followers reject writes from old terms. This prevents a stale leader from corrupting data after it comes back.

This is a simplified version of what Raft calls “terms.”

BUILD

Add a term field to your cluster state. Increment it on every election. Include the term in every replicated write. Followers check: if the write's term is older than my current term, reject it with 403 Forbidden. Log the rejection.

MASTERY CHECK — Your partner verifies

Your partner brings back the old leader (which still thinks it's leader with term=1 while the cluster is on term=2). The old leader tries to replicate a write. Verify that followers reject it. Explain: what would happen without term-based fencing?

REAL-SYSTEM CONNECTION

Read: The Dynamo Paper, Section 4.8 on failure detection. Compare gossip-based detection to your approach.

G4 Split-Brain Prevention

Prerequisites: G3

CONCEPT

Split-brain occurs when a network partition causes two nodes to both believe they are the leader. Both accept writes. When the partition heals, their data diverges. Term-based fencing helps, but the real solution is requiring a quorum — a majority of nodes must agree on the leader.

Read: DDIA Chapter 8, pages 299–302 (the truth is defined by the majority).

BUILD

Modify leader election: a node can only become leader if it receives acknowledgment from a majority of nodes (2 out of 3). If it can't reach a majority, it does NOT promote itself. Test: partition your cluster into [node-1] and [node-2, node-3]. The solo node should NOT become leader.

MASTERY CHECK — Your partner verifies

Explain: why does majority quorum prevent split-brain? What happens with an even number of nodes (e.g., 4)? Why are clusters typically odd-numbered?

G5 Chaos Testing Script

Prerequisites: G4

CONCEPT

A chaos test automates failure injection: kill random nodes, introduce delays, verify data consistency across survivors. This is not a one-time exercise — it's a tool you'll use every time you add a feature. This is the beginning of your chaos harness for incident drills.

BUILD

Write chaos.py: start 3 nodes, write 100 key-value pairs, then randomly kill one node every 10 seconds for 1 minute. After each kill, verify data consistency: all data that was confirmed replicated (i.e., the follower's sequence number had caught up) must be accessible from surviving nodes. Data written to the leader but not yet replicated is EXPECTED to be lost — this is the trade-off of async replication (see F3). Report: how many entries were lost in the replication window, and verify this matches the replication lag you measured in F4.

MASTERY CHECK — Your partner verifies

Run chaos.py 5 times. It should pass every time. If it fails, fix the underlying issue and run it again. Your partner reviews the chaos script: are there failure scenarios it doesn't test?

Domain H: Consistency Models

What does “consistent” actually mean? These topics force you to confront the answer.

H1 Eventual vs Strong Consistency

Prerequisites: F4, G4

CONCEPT

Eventual consistency: if no new writes occur, all replicas will eventually converge to the same value. Strong consistency (linearizability): every read returns the most recent write. Your async-replicated system provides eventual consistency.

Read: DDIA Chapter 9, pages 321–332 (linearizability).

BUILD

Write a test that demonstrates eventual consistency: write to leader, immediately read from follower. Repeat 1000 times. Count how many reads return stale data. Then add a `sleep(0.1)` before the read and measure again. You’re measuring the “eventual” in eventual consistency.

MASTERY CHECK — Your partner verifies

Explain: what does “eventual” mean precisely? Is there a time bound? Give a real-world example where eventual consistency is acceptable and one where it isn’t.

REAL-SYSTEM CONNECTION

Read: The Dynamo Paper, Section 4 on versioning. Understand why Amazon chose eventual consistency for the shopping cart.

H2 Read-Your-Writes Consistency

Prerequisites: H1

CONCEPT

A user writes `A=1` and then reads `A`. They should see 1, even if the read goes to a different replica. Read-your-writes guarantees this. Implementation: after a write, subsequent reads from the same client go to the leader (or a replica known to be up-to-date).

Read: DDIA Chapter 5, pages 162–164.

BUILD

Track the sequence number of each client’s last write (in a dictionary keyed by client IP for now — in production, you’d use a session token or consistency token instead, since IPs are unstable and often shared behind NATs/proxies). When a read comes in, check: has this client written recently? If so,

route the read to the leader or a follower with a sufficiently high sequence number. Otherwise, any follower is fine.

MASTERY CHECK — Your partner verifies

Your partner does: PUT key=hello to the leader, then immediately GET key from a follower. With read-your-writes enabled, it returns “hello”. Disable it, and the read sometimes returns “not found.” Explain why.

H3 Measuring Consistency Violations

Prerequisites: H2

CONCEPT

You can't manage what you don't measure. Write a consistency checker: send writes and reads simultaneously across nodes and record any stale reads. This gives you a concrete number: “X% of reads are stale within Y ms of a write.”

This is what your /metrics should expose.

BUILD

Write a test script: 10 threads write to the leader continuously while 10 threads read from followers. For each read, check if the value is the latest written. Record: total reads, stale reads, max staleness (in sequence numbers). Add a stale_read_count metric to your /metrics endpoint.

MASTERY CHECK — Your partner verifies

State the number: “Under load, X% of reads are stale, with a maximum lag of Y sequence numbers.” Your partner should believe this because you measured it.

R3 Replication & Failure Retrieval

TIMED RETRIEVAL — No notes, no code reference

15-minute time limit. No notes.

Task 1: Explain the difference between synchronous and asynchronous replication. State one advantage and one disadvantage of each.

Task 2: Explain split-brain. Draw the scenario and explain how quorum prevents it.

Task 3: What is replication lag? How did you measure it? What was the number?

All answers must be from memory. If you struggle, revisit F2–F4 and G4.

Domain I: Partitioning

Your data is replicated everywhere. Now partition it so each node owns a subset.

I1 Hash-Based Partitioning

Prerequisites: H1

CONCEPT

Hash each key and assign it to a node based on the hash modulo N . **CRITICAL:** Python's built-in `hash()` is salted per-process (`PYTHONHASHSEED`), so two different nodes will compute different hashes for the same key. You **MUST** use a deterministic hash like `hashlib.sha256`. This is the single most important implementation detail in distributed partitioning.

Read: DDIA Chapter 6, pages 199–204 (partitioning by hash of key).

BUILD

Implement `partition_for_key(key, num_nodes)` using `int(hashlib.sha256(key.encode()).hexdigest(), 16) % num_nodes`. Verify: run the function on two separate Python processes with the same key. Both must return the same node. Then test with Python's built-in `hash()` — it will return **DIFFERENT** results across processes. This is the bug you'd ship in production if you didn't know about `PYTHONHASHSEED`. Distribute 1000 keys across 3 nodes. Print how many keys each node owns. Then change to 4 nodes and print how many keys moved.

MASTERY CHECK — Your partner verifies

State the number of keys that moved. Explain: why is this a problem? What does “moving a key” actually involve in practice (network transfer, downtime, rebalancing)?

I2 Consistent Hashing — The Hash Ring

Prerequisites: I1

CONCEPT

Consistent hashing maps both keys and nodes onto a ring (0 to 2^{32}). Each key belongs to the first node clockwise from its position. Adding a node only moves keys between the new node and its neighbor. On average, only K/N keys move.

Read: DDIA Chapter 6, pages 204–207.

BUILD

Implement a HashRing class: `add_node(node_id)`, `remove_node(node_id)`, `get_node(key)`. Use `hashlib.md5` or `sha256` for deterministic hashing. Test with 1000 keys and 3 nodes. Add a 4th node. Print how many keys moved. Compare to the modulo approach.

MASTERY CHECK — Your partner verifies

State the numbers side by side: modulo moved X keys, consistent hashing moved Y keys. Explain the algorithm to your partner using a drawn ring.

REAL-SYSTEM CONNECTION

Read: Cassandra's documentation on consistent hashing ([docs.datastax.com](https://docs.datastax.com/en/cassandra/3.0/cassandra/architecture/consistentHashing.html)). Note how they use virtual nodes (vnodes) to improve load distribution. Explain why.

I3 Request Routing and Proxying

Prerequisites: I2

CONCEPT

A client can send a request to any node. That node checks: do I own this key? If yes, handle it. If no, proxy the request to the correct owner. The client doesn't need to know the partition map. This is a routing layer.

BUILD

When a request arrives, compute the partition owner using your hash ring. If this node owns the key, handle it locally. If not, use `send_request()` to forward to the correct node and return that node's response to the client. Log every proxy decision.

MASTERY CHECK — Your partner verifies

Your partner sends 20 requests to random nodes for keys owned by different nodes. All requests succeed with correct responses. Trace one proxied request and explain the full path.

14 Partitioning + Replication Combined

Prerequisites: I3, F3

CONCEPT

Each partition should be replicated for fault tolerance. The primary owner handles writes, and the next node on the ring holds a replica. If the primary dies, the replica can serve reads and accept writes after promotion.

Read: DDIA Chapter 6, pages 209–211 (partitioning and replication).

BUILD

For each key, the hash ring determines the primary node. The next node clockwise is the replica. Writes go to the primary, which replicates to the replica asynchronously. Reads can go to either. Implement this.

MASTERY CHECK — Your partner verifies

Kill the primary owner of a partition. Verify that the replica still serves reads for that partition's keys. Explain: what happens to writes for that partition while the primary is down?

REAL-SYSTEM CONNECTION

Read: The Dynamo Paper, Section 4.3 on replication. Compare the “preference list” concept to your approach.

15 Adding and Removing Nodes

Prerequisites: I4

CONCEPT

When a new node joins, it takes ownership of some key ranges. Keys in those ranges must be transferred from the previous owner. When a node leaves, its key ranges are redistributed. This is called rebalancing.

Read: DDIA Chapter 6, pages 209–215 (rebalancing partitions).

BUILD

Implement `add_node`: (1) update the hash ring, (2) identify which keys moved to the new node, (3) transfer those keys from the old owner. Implement `remove_node`: (1) update the ring, (2) transfer keys to new owners. Test by adding a 4th node to a 3-node cluster with 1000 keys.

MASTERY CHECK — Your partner verifies

Your partner adds and removes a node. After each operation, verify that all keys are still accessible and no data was lost. State: how many keys were transferred during each operation?

AUTOMATICITY GATE: Diagnose and Fix a Cluster Failure

Your partner sets up a 3-node cluster and injects a failure (they choose from: killed node, network partition, split-brain, corrupted log entry). You must diagnose and fix it using only your logs, metrics, and debugging skills. No hints from your partner.

GATE REQUIREMENTS

Your partner prepares the failure scenario in advance. You don't know what it is.

Timer starts when you're told "something is wrong." You have 30 minutes.

You must: (1) identify symptoms from logs and metrics, (2) form a hypothesis, (3) verify the hypothesis, (4) fix the issue or explain exactly how you would fix it.

Write a brief incident postmortem: what happened, how you diagnosed it, what the fix was.

If you cannot diagnose within 30 minutes, your partner reveals the issue. You revisit the relevant topics and attempt a new scenario the following week.

Domain J: Observability

You've already been adding basic logging and metrics. These topics formalize your observability stack.

J1 Structured JSON Logging

Prerequisites: Gate 2

CONCEPT

Replace all `print()` calls with structured JSON logs. Every log entry has: timestamp (ISO 8601), `node_id`, level (INFO/WARN/ERROR), event (what happened), and context (relevant data). Structured logs are machine-parseable and searchable.
Read: DSO, chapters 1–2 (what is observability?).

BUILD

Build a `log(level, event, **context)` function that outputs JSON to stdout. Replace every `print()` in your codebase. Standardize event names (e.g., "request_received", "replication_sent", "health_check_failed"). Log every significant event.

MASTERY CHECK — Your partner verifies

Your partner greps your logs for a specific trace. Can they reconstruct what happened? If your logs tell a coherent story, you pass.

J2 Metrics — Counters and Histograms

Prerequisites: J1

CONCEPT

A counter is a value that only goes up (total requests, total errors). A histogram tracks the distribution of a value (request latency). From a histogram you can derive percentiles. Your `/metrics` endpoint should expose both.
Read: DSO, chapter 4 (metrics).

BUILD

Build a Metrics class with: `increment(name)`, `observe(name, value)`. Compute: count, sum, average, p50, p95, p99 for histograms. Track: `request_count`, `request_latency`, `replication_lag`, `error_count`, `active_connections`. Expose via GET `/metrics` as JSON.

MASTERY CHECK — Your partner verifies

Under load, your /metrics endpoint returns meaningful data. Your partner can identify from the metrics alone whether the system is healthy, degraded, or failing.

REAL-SYSTEM CONNECTION

Read: “How Uber Built its Observability Platform” (newsletter.pragmaticengineer.com). Compare your metrics to what Uber’s M3 tracks.

J3 Distributed Request Tracing

Prerequisites: J2

CONCEPT

Assign each client request a unique `trace_id` (UUID). Pass it in a header (X-Trace-ID) through all inter-node calls. Every log entry for that request includes the `trace_id`. When a request spans 3 nodes, all log entries share the same `trace_id`.

Read: DSO, chapter 5 (distributed tracing).

BUILD

Generate a `trace_id` when a request first enters the system. Include it in all internal HTTP calls as a header. Include it in every log entry. Write a script that searches logs across all nodes for a specific `trace_id` and reconstructs the full request path.

MASTERY CHECK — Your partner verifies

Your partner gives you a `trace_id`. You reconstruct the full path of the request across nodes, including timing at each hop. Explain: how is this similar to Uber’s Jaeger?

REAL-SYSTEM CONNECTION

Read: Uber’s Jaeger distributed tracing blog (uber.com/blog/optimizing-observability).

J4 Terminal Dashboard

Prerequisites: J3

CONCEPT

A dashboard polls /metrics on all nodes and displays a live view of the cluster’s health. It should show: per-node throughput, latency percentiles, replication lag, error rates, and node status. Update every second.

This is your “single pane of glass.”

BUILD

Write a Python script using `os.system('clear')` and formatted print statements that polls `/metrics` on all nodes every second. Display: node status (UP/DOWN), requests/sec, p99 latency, replication lag, error rate. Use colors (green/yellow/red) for status.

MASTERY CHECK — Your partner verifies

Start the dashboard, run a load test, kill a node. Your partner watches the dashboard and should be able to tell you exactly when the node died, how the system responded, and whether any errors occurred — all from the dashboard alone.

Domain K: Performance

Find your system's limits. Fix the bottleneck. Know your numbers.

K1 Load Testing with Locust

Prerequisites: J4

CONCEPT

Locust is a Python load testing framework. Define user behaviors (read/write patterns), ramp up concurrent users, and observe how your system responds. This is where you're allowed to use a third-party library.

Install with: `pip install locust`

BUILD

Write a `locustfile.py` that defines two user types: readers (80% of users, do GETs) and writers (20%, do PUTs). Ramp from 10 to 200 concurrent users over 5 minutes. Watch your terminal dashboard during the test. Record the throughput and latency curves.

MASTERY CHECK — Your partner verifies

State: at what concurrency level does p99 latency exceed 100ms? At what level does the error rate exceed 1%? These are your system's operating limits.

K2 Profiling with cProfile

Prerequisites: K1

CONCEPT

cProfile instruments your Python code and tells you where time is actually spent. Don't guess — measure. The bottleneck is rarely where you think it is.

Run with: `python -m cProfile -s cumulative your_server.py`

BUILD

Profile your server under load (run locust in another terminal). Identify the top 5 functions by cumulative time. Is the bottleneck socket I/O? HTTP parsing? Disk writes? Lock contention? Dictionary operations?

MASTERY CHECK — Your partner verifies

Show your partner the profiling output. Explain each of the top 5 functions and why it's taking time. Predict which one is the real bottleneck and why.

K3 Optimizing One Bottleneck

Prerequisites: K2

CONCEPT

Pick the biggest bottleneck from profiling. Optimize it. Measure before and after. The optimization should be data-driven, not a guess. Common optimizations: buffer sizes, fewer fsync calls, connection pooling, avoiding unnecessary copies.

One targeted optimization beats ten random ones.

BUILD

Implement the optimization. Run the exact same load test as K1. Compare: throughput before vs after, p99 before vs after. If the improvement is less than 10%, the bottleneck is elsewhere.

MASTERY CHECK — Your partner verifies

State: “I optimized [specific thing] and throughput improved from X to Y req/sec.” If throughput barely changed, explain why (the bottleneck shifted elsewhere).

REAL-SYSTEM CONNECTION

Read: Prometheus documentation on metric types. Compare their model to your /metrics.

R4 Full Stack Retrieval

TIMED RETRIEVAL — No notes, no code reference

20-minute time limit. No notes.

Task 1: Explain how a PUT request flows through your distributed system, from the client's `socket.connect()` through routing, leader processing, replication, and response. Include every network hop.

Task 2: What are your system's performance numbers? Throughput, p99, bottleneck, at what concurrency it degrades.

Task 3: Describe three failure scenarios your chaos test covers and how your system handles each. This retrieval covers ALL domains. If you struggle with any part, revisit those topics.

Domain L: Code Quality

By now your codebase is messy. You've earned the right to refactor it — because you feel the pain.

L1 Strategy Pattern — Storage Backends

Prerequisites: K3

CONCEPT

Define an abstract `StorageBackend` with `put(key, value)`, `get(key)`, and `close()` methods. Your append-only log is one implementation. Create an `InMemoryBackend` for testing. Both implement the same interface. Your server doesn't know which one it's using.

This is the Strategy pattern.

BUILD

Create an ABC (Abstract Base Class) or `typing.Protocol` for `StorageBackend`. Refactor your log storage to implement it. Create an `InMemoryBackend`. Your server accepts a backend parameter. Run your entire test suite with both backends.

MASTERY CHECK — Your partner verifies

Your partner asks: if you had to replace your storage with SQLite, how many files would you change? If the answer is more than 2, your abstraction is leaking.

L2 Observer Pattern — Write Notifications

Prerequisites: L1

CONCEPT

When a write happens, multiple things need to know: the replication manager, the metrics collector, the logger. The Observer pattern decouples the write path from all these consumers. The storage engine doesn't know about replication.

This is the Observer pattern.

BUILD

Create a WriteObserver protocol with `on_write(key, value, sequence_num)`. Replication, metrics, and logging each implement it. The storage engine maintains a list of observers and notifies them on every write.

MASTERY CHECK — Your partner verifies

Add a new observer (e.g., an audit logger) without modifying the storage engine code. If you can't, the decoupling isn't clean enough.

L3 Module Boundaries

Prerequisites: L2

CONCEPT

Separate your code into clear modules: transport (TCP/HTTP), storage, replication, cluster, observability. Each module has a clear public interface and minimal dependencies on others. Imports should flow in one direction.

Clean boundaries make the system testable and changeable.

BUILD

Reorganize your codebase into separate Python modules. Each module's `__init__.py` exports only its public interface. Draw the dependency graph between modules. Verify that there are no circular dependencies.

MASTERY CHECK — Your partner verifies

Describe each module's public interface in one sentence. Your partner should understand the system's architecture from these sentences alone.

L4 Testing

Prerequisites: L3

CONCEPT

Unit test each module independently: storage operations, HTTP parsing, consistent hashing, cluster state. Integration test replication and failover. Use your InMemoryBackend for fast unit tests. Use the real backend for integration tests.

Tests are documentation that runs.

BUILD

Write tests for: (1) storage put/get/crash-recovery, (2) HTTP parser with edge cases, (3) consistent hashing key distribution, (4) replication with simulated node failure. Aim for the critical paths, not 100% coverage.

MASTERY CHECK — Your partner verifies

Run your test suite. All tests pass. Your partner adds a bug to one module. The tests catch it.

Domain M: System Design Translation

Translate your hands-on knowledge into interview-ready reasoning.

M1 Architecture Documentation

Prerequisites: L4

CONCEPT

Write a README that a senior engineer can read in 5 minutes and understand your system. Include: architecture diagram, design decisions with trade-offs, performance numbers, failure modes and how they're handled. This IS your interview artifact.

This is the most valuable thing you'll produce.

BUILD

Write the README. Include a text-based architecture diagram. For each design decision (e.g., "why async replication?"), state the alternatives you considered and why you chose this one. Include your measured performance numbers.

MASTERY CHECK — Your partner verifies

Your partner reads the README cold. Can they explain your system's architecture back to you? If yes, the README works.

M2 Design Exercise: Rate Limiter

Prerequisites: M1

CONCEPT

Design a distributed rate limiter. Think from first principles: where does the count live? How do you handle clock skew? What happens if the rate limiter node fails? What consistency model do you need?

Apply everything you learned.

BUILD

Write a 1-page design document for a distributed rate limiter. Specify: data structure, storage, partitioning, failure handling, consistency model, metrics. Compare to how Redis implements rate limiting.

MASTERY CHECK — Your partner verifies

Present your design to your partner in 10 minutes. They play interviewer and ask follow-up questions. You answer from first principles, not memorized patterns.

M3 Design Exercise: Message Queue

Prerequisites: M1

CONCEPT

Design a distributed message queue. How does it differ from your KV store? Ordering matters. Delivery guarantees (at-most-once, at-least-once, exactly-once) are hard. Consumer groups add complexity.

Notice how many of these problems you already solved.

BUILD

Write a 1-page design. Specify: how messages are stored (compare to your log), how consumers track their position (compare to your sequence numbers), what happens when a consumer crashes, what happens when a broker crashes.

MASTERY CHECK — Your partner verifies

Present to your partner. Compare your design to Kafka's architecture. Where do they align? Where do they differ? Why?

M4 Mock Interview

Prerequisites: M2, M3

CONCEPT

Each partner picks a system design question for the other. 35 minutes to design, 10 minutes for feedback. This is the final test: can you reason about a system you've never built, from first principles?

Record it if possible. Watch it back.

BUILD

Run the mock interview. The interviewer pushes on: failure modes, trade-offs, performance numbers, alternative approaches. The candidate answers by reasoning from their experience building the KV store.

MASTERY CHECK — Your partner verifies

After the interview, each person writes: (1) what went well, (2) where they got stuck, (3) what they would study next. These gaps become your next learning plan.

R5 Distributed Systems Retrieval

TIMED RETRIEVAL — No notes, no code reference

20-minute time limit. No notes.

Task: Explain consistent hashing. Draw a hash ring with 4 nodes and show what happens when you add a 5th. State how many keys move and why.

Task: Explain the CAP theorem using your own system as the example. Which side did you choose? Why?

Task: Compare your replication approach to MySQL's binlog replication. Name two differences.

R6 Full Curriculum Retrieval

TIMED RETRIEVAL — No notes, no code reference

30-minute time limit. No notes. This is the final retrieval.

Task 1: Explain your system's architecture end-to-end in 3 minutes. Include: request flow, storage, replication, partitioning, failure handling, observability.

Task 2: Someone asks "Why not just use Redis?" Give a 2-minute answer.

Task 3: Design a distributed URL shortener on a whiteboard. 15 minutes. Include: partitioning strategy, replication, failure handling, metrics.

If you can do all three, you've internalized the material. If not, you know exactly which domains to revisit.

Recommended Topic Path (Interleaved)

This is the order you should follow. It interleaves topics across domains while respecting prerequisites. Retrieval exercises and gates are inserted at specific points. Each line is one topic session.

Track your progress: mark each topic as Not Started → In Progress → Built → Mastery Verified.

1. A1 Socket Creation and Binding
2. A2 Listening and Accepting
3. A3 Send and Receive Byte Streams
4. A4 Line-Based Message Framing
5. A5 Connection Lifecycle and Cleanup
6. B1 HTTP Request Line Parsing
7. A6 Socket Timeouts and Error Handling
8. B2 HTTP Header Parsing
9. B3 HTTP Body Handling
10. B4 HTTP Response Construction
11. B5 Routing
12. B6 KV Store HTTP Interface
13. C1 File I/O Fundamentals
14. C2 Append-Only Log
15. C3 In-Memory Index Over Log
16. C4 Crash Recovery
17. C5 fsync and Durability
18. D1 The GIL
19. C6 Log Compaction
20. D2 Thread-Per-Connection
21. D3 Thread Pool
22. D4 Shared State and Race Conditions
23. D5 Read-Write Locks
24. D6 Measuring Concurrent Performance
25. ⚠ **GATE 1** Rebuild Single-Node KV Store from Scratch
26. E1 Multi-Node Configuration
27. E2 Node-to-Node HTTP Client
28. **R1** TCP & HTTP Retrieval
29. E3 Health Checking
30. E4 Cluster State Tracking
31. F1 Leader Designation
32. F2 Synchronous Replication
33. J1 Structured JSON Logging
34. F3 Asynchronous Replication
35. F4 Measuring Replication Lag
36. **R2** Storage & Concurrency Retrieval
37. F5 Follower Catch-Up

- 38. G1 Leader Failure Detection
- 39. J2 Metrics — Counters and Histograms
- 40. G2 Basic Leader Election
- 41. G3 Term Numbers and Fencing
- 42. G4 Split-Brain Prevention
- 43. G5 Chaos Testing Script
- 44. H1 Eventual vs Strong Consistency
- 45. J3 Distributed Request Tracing
- 46. H2 Read-Your-Writes
- 47. H3 Measuring Consistency Violations
- 48. **R3** Replication & Failure Retrieval
- 49. I1 Hash-Based Partitioning
- 50. I2 Consistent Hashing
- 51. J4 Terminal Dashboard
- 52. I3 Request Routing
- 53. I4 Partitioning + Replication
- 54. I5 Adding/Removing Nodes
- 55.  **GATE 2** Diagnose and Fix a Cluster Failure
- 56. K1 Load Testing with Locust
- 57. K2 Profiling with cProfile
- 58. K3 Optimizing One Bottleneck
- 59. **R4** Full Stack Retrieval
- 60. L1 Strategy Pattern
- 61. L2 Observer Pattern
- 62. L3 Module Boundaries
- 63. L4 Testing
- 64. M1 Architecture Documentation
- 65. **R5** Distributed Systems Retrieval
- 66. M2 Design: Rate Limiter
- 67. M3 Design: Message Queue
- 68. M4 Mock Interview
- 69. **R6** Full Curriculum Retrieval

Partner & Blogging Protocol

Mastery Verification

After completing each topic's BUILD task, you notify your partner. They run the MASTERY CHECK — asking you to explain or demonstrate without notes. They mark the topic as Mastery Verified on your tracking board, or flag it for revisiting.

Be rigorous. “I sort of get it” is not mastery. “I can explain it and demonstrate it from memory” is mastery.

Incident Drills

After Gate 2, run incident drills every 2 weeks using the Claude Code model discussed earlier. Your partner's Claude Code instance designs and injects failures. You diagnose under time pressure. Write postmortems.

Blogging

After every 15–20 topics, write one blog post. Structure:

- **What you built** (the system features from those topics)
- **What surprised you** (the “I didn't expect that” moments)
- **How your implementations differed** (compare with your partner)
- **Real-system connection** (how does MySQL/Cassandra/Uber solve this?)
- **Retrieval results** (which concepts stuck, which decayed)

Target: 4–5 posts over the full curriculum.

Final Notes

This curriculum has no timeline. Some people will finish in 14 weeks. Others in 24. The pace of mastery is the pace of learning. Rushing through topics without genuine mastery is the single most destructive thing you can do.

The mastery checks are the curriculum. The BUILD tasks produce code. The mastery checks produce understanding. If you skip mastery verification to move faster, you are learning nothing. You are just typing.

Your partner is not optional. Self-assessed mastery is unreliable. You need someone who will look you in the eye and say “explain split-brain to me right now without notes” and not let you move on until you can.

The retrieval exercises will feel hard. That’s the point. Retrieval difficulty is the mechanism by which long-term memory is strengthened. If a retrieval exercise feels easy, the knowledge has already stuck. If it feels hard, you’re catching decay exactly when you should.

After this curriculum, you will be an engineer who can reason about systems from first principles. Not because you memorized patterns, but because you built a system from raw sockets to distributed consensus and can explain every layer from memory.